M.SRI RAM 192311120

SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA43

COURSE OUTCOME COVERED

CO-2 : Apply CSS layout and styling techniques to create visually appealing and responsive web interfaces, and use JavaScript to enable interactive client-side behavior. (L3)

Assessment Tool : Integrated Experiments

Weightage: 40%

QUESTIONS: **Total Marks: 30**

Experiment 1: Responsive Layout Design

Suppose that a retail website is not responsive across mobile and tablet devices, causing misalignment of content and improper spacing.
(a) Analyze the possible reasons for lack of responsiveness in the existing layout.
(b) Design a responsive webpage layout using **CSS Flexbox or Grid**, ensuring proper alignment of header, navigation, content, and footer.
(c) Demonstrate how the layout adapts to **different screen sizes (mobile, tablet, desktop)** using media queries.
(d) Implement spacing, alignment, and distribution techniques to improve visual consistency.
(e) Evaluate the effectiveness of the responsive design and suggest further improvements.

Experiment 2: Form Validation using JavaScript

Suppose that a registration form accepts invalid email and phone number inputs, leading to incorrect data submission.
(a) Analyze the issues in the existing form validation mechanism.
(b) Design input fields for email and phone number with appropriate HTML attributes.
(c) Implement **JavaScript validation using Regular Expressions** for both email and phone number.

(d) Develop a mechanism to display **real-time error messages** for invalid inputs.
(e) Evaluate the validation system and suggest enhancements for better user experience and security.

Experiment 3: CSS Layout Debugging

Suppose that a webpage layout overlaps when the browser window is resized, affecting usability.
(a) Analyze the CSS properties responsible for layout overlapping issues.
(b) Examine the role of the **CSS box model (margin, border, padding, content)** in layout design.
(c) Debug the layout using appropriate **positioning techniques (relative, absolute, flex, grid)**.
(d) Modify the CSS to ensure proper alignment and prevent overlapping during resizing.
(e) Evaluate the corrected layout and suggest best practices for maintaining consistency.

Code of Conduct:

I **M.SRIRAM / 192311120** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: M.SRIRAM

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools

Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

M.SRI RAM 192311120

ANSWER PAPER

Internet Programming (CSA43) – CO-2 Integrated Experiments

Experiment 1: Responsive Layout Design

(a) Reasons for lack of responsiveness

- **Missing viewport meta tag** – mobile browsers render the page at ~980px and shrink it, so text becomes tiny.
- **Fixed pixel widths** (e.g. `width:960px`) instead of %, `fr`, `rem` or `max-width`, causing horizontal scrolling.
- **No media queries**, so the same desktop layout is forced onto tablet and mobile screens.
- **Floats / absolute positioning** used for the page structure instead of Flexbox or Grid.
- **Images without `max-width:100%`**, overflowing their containers.
- **Inconsistent spacing** (hard-coded margins, no `box-sizing:border-box`), which misaligns content.

(b) Responsive layout using Flexbox and Grid (Code: exp1.html)

Header and navigation use **Flexbox**; the page body (sidebar + products) and the product cards use **CSS Grid**; the footer is Flexbox. The page is a flex column so the footer stays at the bottom.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Experiment 1 - Responsive Retail Layout</title>
<style>
  /* ---------- Reset & tokens ---------- */
  *, *::before, *::after { box-sizing: border-box; margin: 0; padding: 0; }
  :root {
    --brand: #1e3a8a;
    --accent: #f59e0b;
    --bg: #f3f4f6;
    --card: #ffffff;
    --text: #1f2937;
    --gap: 1rem;
  }
  body {
    font-family: Arial, Helvetica, sans-serif;
    color: var(--text);
    background: var(--bg);
    line-height: 1.5;
    min-height: 100vh;
    display: flex;
    flex-direction: column;       /* header / nav / main / footer stack */
  }
  img { max-width: 100%; height: auto; display: block; }

  /* ---------- Header ---------- */
  header {
    background: var(--brand);
    color: #fff;
    padding: var(--gap) 1.5rem;
    display: flex;
    justify-content: space-between;
    align-items: center;
    flex-wrap: wrap;
    gap: var(--gap);
  }
  header h1 { font-size: 1.5rem; }
  header input {
    padding: .5rem .75rem;
    border: none;
```

```
  border-radius: 4px;
  min-width: 180px;
  flex: 1 1 180px;
  max-width: 320px;
}

/* ---------- Navigation (Flexbox) ---------- */
nav { background: #1e40af; }
nav ul {
  list-style: none;
  display: flex;
  flex-direction: column;       /* mobile: stacked */
}
nav a {
  display: block;
  padding: .75rem 1.5rem;
  color: #fff;
  text-decoration: none;
  border-bottom: 1px solid rgba(255,255,255,.15);
}
nav a:hover { background: var(--accent); color: #000; }

/* ---------- Main content (CSS Grid) ---------- */
main {
  flex: 1;
  width: 100%;
  max-width: 1200px;
  margin: 0 auto;
  padding: 1.5rem var(--gap);
  display: grid;
  grid-template-columns: 1fr;   /* mobile: 1 column */
  gap: 1.5rem;
}
.products {
  display: grid;
  grid-template-columns: 1fr;
  gap: var(--gap);
}
.card {
  background: var(--card);
  border-radius: 8px;
  padding: var(--gap);
  box-shadow: 0 1px 4px rgba(0,0,0,.12);
  display: flex;
  flex-direction: column;
  gap: .5rem;
}
.card .thumb {
  background: linear-gradient(135deg, #93c5fd, #c7d2fe);
  height: 110px;
  border-radius: 6px;
  display: flex; align-items: center; justify-content: center;
  font-size: 2rem;
}
.card .price { font-weight: bold; color: var(--brand); }
.card button {
  margin-top: auto;
  padding: .5rem;
  background: var(--accent);
  border: none;
  border-radius: 4px;
  font-weight: bold;
  cursor: pointer;
}
aside.sidebar {
  background: var(--card);
  border-radius: 8px;
  padding: var(--gap);
  box-shadow: 0 1px 4px rgba(0,0,0,.12);
}
aside h2, .products-title { margin-bottom: .5rem; }

/* ---------- Footer ---------- */
footer {
  background: #111827;
  color: #d1d5db;
  padding: 1.5rem;
  display: flex;
  flex-direction: column;
```

```
    gap: var(--gap);
    text-align: center;
  }

  /* ---------- Breakpoints (mobile-first) ---------- */
  /* Tablet */
  @media (min-width: 600px) {
    .products { grid-template-columns: repeat(2, 1fr); }
    nav ul { flex-direction: row; flex-wrap: wrap; }
    nav a { border-bottom: none; }
    footer { flex-direction: row; justify-content: space-between; text-align: left; }
  }
  /* Desktop */
  @media (min-width: 992px) {
    main { grid-template-columns: 240px 1fr; }     /* sidebar + content */
    .products { grid-template-columns: repeat(3, 1fr); }
    nav ul { justify-content: center; }
  }
  /* Large desktop */
  @media (min-width: 1280px) {
    .products { grid-template-columns: repeat(4, 1fr); }
  }
</style>
</head>
<body>
  <header>
    <h1>&#128722; ShopEasy</h1>
    <input type="search" placeholder="Search products...">
  </header>

  <nav>
    <ul>
      <li><a href="#">Home</a></li>
      <li><a href="#">Electronics</a></li>
      <li><a href="#">Fashion</a></li>
      <li><a href="#">Home &amp; Kitchen</a></li>
      <li><a href="#">Contact</a></li>
    </ul>
  </nav>

  <main>
    <aside class="sidebar">
      <h2>Filters</h2>
      <p>Price: Under &#8377;5,000</p>
      <p>Brand: All</p>
      <p>Rating: 4&#9733; &amp; up</p>
    </aside>

    <section>
      <h2 class="products-title">Featured Products</h2>
      <div class="products">
        <article class="card"><div class="thumb">&#127911;</div><h3>Headphones</h3><p
class="price">&#8377;1,999</p><button>Add to Cart</button></article>
        <article class="card"><div class="thumb">&#8986;</div><h3>Smart Watch</h3><p
class="price">&#8377;3,499</p><button>Add to Cart</button></article>
        <article class="card"><div class="thumb">&#128085;</div><h3>T-Shirt</h3><p
class="price">&#8377;599</p><button>Add to Cart</button></article>
        <article class="card"><div class="thumb">&#128095;</div><h3>Sneakers</h3><p
class="price">&#8377;2,499</p><button>Add to Cart</button></article>
        <article class="card"><div class="thumb">&#128247;</div><h3>Camera</h3><p
class="price">&#8377;4,799</p><button>Add to Cart</button></article>
        <article class="card"><div class="thumb">&#127890;</div><h3>Backpack</h3><p
class="price">&#8377;999</p><button>Add to Cart</button></article>
      </div>
    </section>
  </main>

  <footer>
    <span>&copy; 2026 ShopEasy</span>
    <span>Privacy &bull; Terms &bull; Support</span>
  </footer>
</body>
</html>
```

(c) Adaptation to screen sizes (media queries)

Screen	Breakpoint	Navigation	Product columns	Sidebar

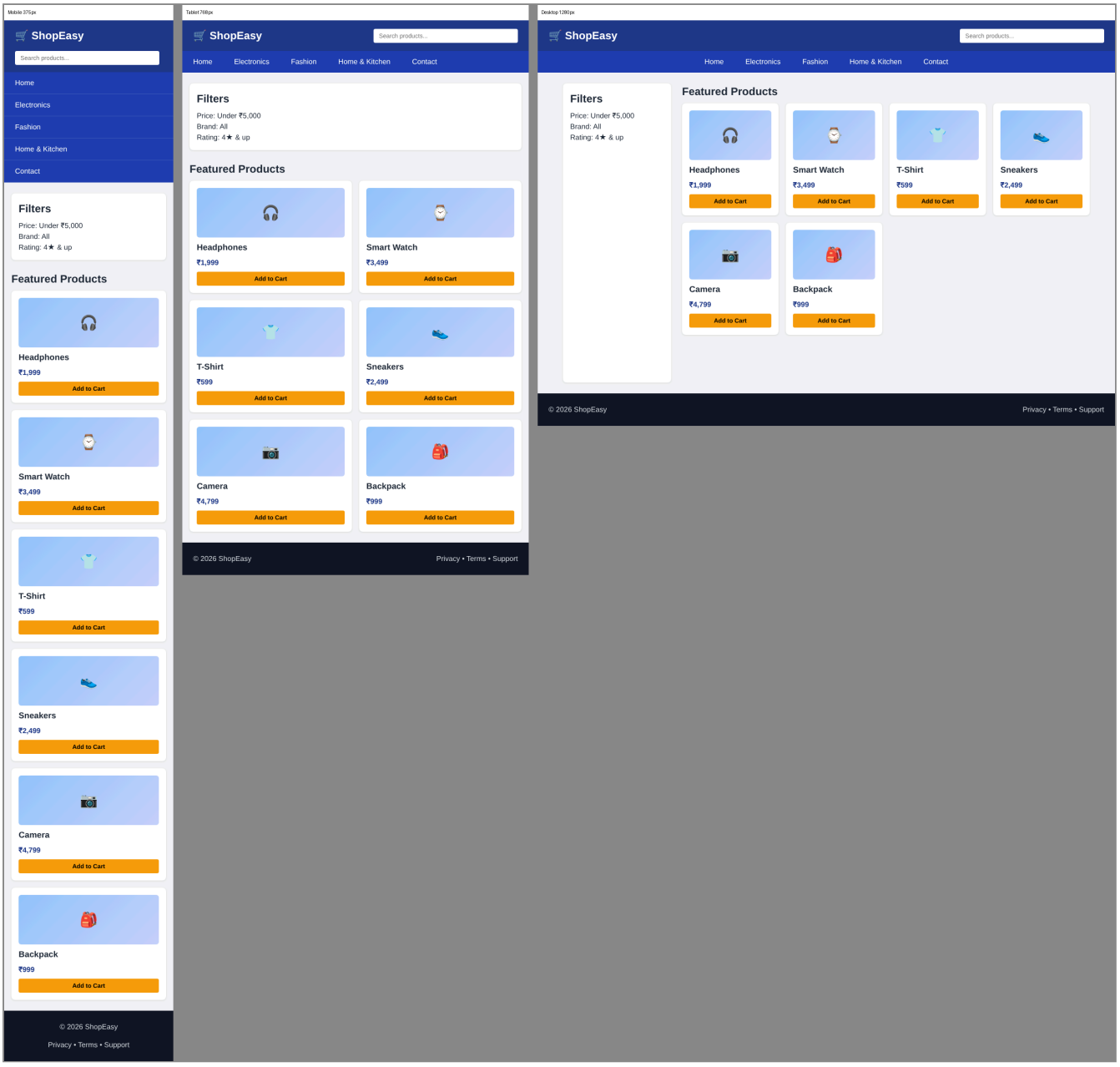
Mobile	< 600px	Stacked vertically	1	Above products
Tablet	≥ 600px	Horizontal row	2	Above products
Desktop	≥ 992px	Centered row	3 (4 at ≥ 1280px)	Left column (240px)

(d) Spacing, alignment and distribution

- `box-sizing:border-box` and a CSS reset for consistent sizing.
- `gap` (1rem / 1.5rem) in Flex and Grid instead of margins between items.
- `justify-content:space-between` in the header and footer; `align-items:center` for vertical alignment.
- `max-width:1200px; margin:0 auto` keeps content centred on wide screens.
- Cards use a column flex layout with `margin-top:auto` on the button, so all buttons align at the bottom.

Sample input and output

Sample input (browser width)	Output
375px	Single column, stacked menu, search box below logo
768px	Two product columns, horizontal menu
1280px	Sidebar + four product columns, centered menu



Output screenshot: Experiment 1 at mobile (375px), tablet (768px) and desktop (1280px)

(e) Evaluation and improvements

- **Effectiveness:** at 375px the page shows one column with a stacked menu, at 768px two product columns with a horizontal menu, and at 1280px a sidebar plus four columns. There is no horizontal scrolling at any width, and the header, navigation, content and footer stay aligned.
- **Improvements:** use `srcset`/lazy-loaded images, a hamburger menu on mobile, `clamp()` for fluid typography, `auto-fit/minmax()` grids, touch targets of at least 44px, and test with Lighthouse on real devices.

Experiment 2: Form Validation using JavaScript

(a) Issues in the existing validation

- The form uses only `type="text"` for every field, so no format checking happens.
- No validation script, or a weak check such as `value != ''`, so values like `abc@` or `12345` are accepted.
- Validation runs only on submit, so the user gets no feedback while typing.
- There is no restriction on phone length or characters (letters and symbols are allowed).
- There is no server-side validation, so invalid data can still reach the database.

(b) Input fields with HTML attributes

Field	HTML attributes	Purpose
Email	`type="email" required maxlength="100" inputmode="email" autocomplete="email"`	Browser email check, mandatory field, email keyboard on mobile
Phone	`type="tel" required maxlength="10" pattern="[6-9][0-9]{9}" inputmode="numeric"`	Exactly 10 digits starting with 6-9, numeric keypad

(c), (d) JavaScript validation with regular expressions and real-time error messages (Code: exp2.html)

Regular expressions: Email `/^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$/`; Phone `/^[6-9][0-9]{9}$/`. Errors are shown on every `input` event and on `blur`.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Experiment 2 - Registration Form Validation</title>
<style>
  * { box-sizing: border-box; }
  body { font-family: Arial, sans-serif; background: #eef2f7; display: flex; justify-content: center;
padding: 2rem 1rem; }
  form { background: #fff; width: 100%; max-width: 420px; padding: 1.5rem; border-radius: 8px; box-shadow:
0 2px 8px rgba(0,0,0,.15); }
  h2 { margin-bottom: 1rem; color: #1e3a8a; }
  .field { margin-bottom: 1rem; }
  label { display: block; font-weight: bold; margin-bottom: .25rem; }
  input { width: 100%; padding: .55rem; border: 2px solid #cbd5e1; border-radius: 4px; font-size: 1rem; }
  input.valid   { border-color: #16a34a; }
  input.invalid { border-color: #dc2626; }
  .error { color: #dc2626; font-size: .85rem; min-height: 1.1rem; margin-top: .25rem; }
  .ok    { color: #16a34a; font-size: .85rem; min-height: 1.1rem; margin-top: .25rem; }
  button { width: 100%; padding: .65rem; background: #1e3a8a; color: #fff; border: none; border-radius:
4px; font-size: 1rem; cursor: pointer; }
  button:disabled { background: #94a3b8; cursor: not-allowed; }
  #result { margin-top: 1rem; font-weight: bold; color: #16a34a; }
</style>
</head>
<body>
<form id="regForm" novalidate>
  <h2>Registration Form</h2>

  <div class="field">
    <label for="name">Full Name</label>
    <input type="text" id="name" name="name" placeholder="Your name" required minlength="3"
autocomplete="name">
    <div class="error" id="nameMsg"></div>
  </div>

  <div class="field">
    <label for="email">Email</label>
    <!-- type=email + required + pattern give built-in browser support -->
    <input type="email" id="email" name="email" placeholder="name@example.com" required
           maxlength="100" autocomplete="email" inputmode="email">
    <div class="error" id="emailMsg"></div>
  </div>
```

```
  <div class="field">
    <label for="phone">Phone Number</label>
    <input type="tel" id="phone" name="phone" placeholder="9876543210" required
           maxlength="10" pattern="[6-9][0-9]{9}" autocomplete="tel" inputmode="numeric">
    <div class="error" id="phoneMsg"></div>
  </div>

  <button type="submit" id="submitBtn" disabled>Register</button>
  <div id="result"></div>
</form>

<script>
  // ---------- Regular expressions ----------
  const NAME_RE  = /^[A-Za-z][A-Za-z .'-]{2,49}$/;
  const EMAIL_RE = /^[A-Za-z0-9._%+-]+@[A-Za-z0-9-]+(\.[A-Za-z0-9-]+)*\.[A-Za-z]{2,}$/;
  const PHONE_RE = /^[6-9][0-9]{9}$/;          // Indian 10-digit mobile

  const fields = {
    name:  { el: document.getElementById('name'),  msg: document.getElementById('nameMsg'),
             re: NAME_RE,  empty: 'Name is required.',
             bad: 'Use at least 3 letters (letters, space, . \' - only).' },
    email: { el: document.getElementById('email'), msg: document.getElementById('emailMsg'),
             re: EMAIL_RE, empty: 'Email is required.',
             bad: 'Enter a valid email, e.g. name@example.com.' },
    phone: { el: document.getElementById('phone'), msg: document.getElementById('phoneMsg'),
             re: PHONE_RE, empty: 'Phone number is required.',
             bad: 'Enter 10 digits starting with 6, 7, 8 or 9.' }
  };
  const btn = document.getElementById('submitBtn');
  const result = document.getElementById('result');

  function validate(key, showEmpty) {
    const f = fields[key];
    const value = f.el.value.trim();
    let ok = false;
    if (value === '') {
      f.msg.textContent = showEmpty ? f.empty : '';
      f.msg.className = 'error';
      f.el.className = showEmpty ? 'invalid' : '';
    } else if (!f.re.test(value)) {
      f.msg.textContent = f.bad;                 // real-time error
      f.msg.className = 'error';
      f.el.className = 'invalid';
    } else {
      f.msg.textContent = '✓ Looks good';
      f.msg.className = 'ok';
      f.el.className = 'valid';
      ok = true;
    }
    return ok;
  }

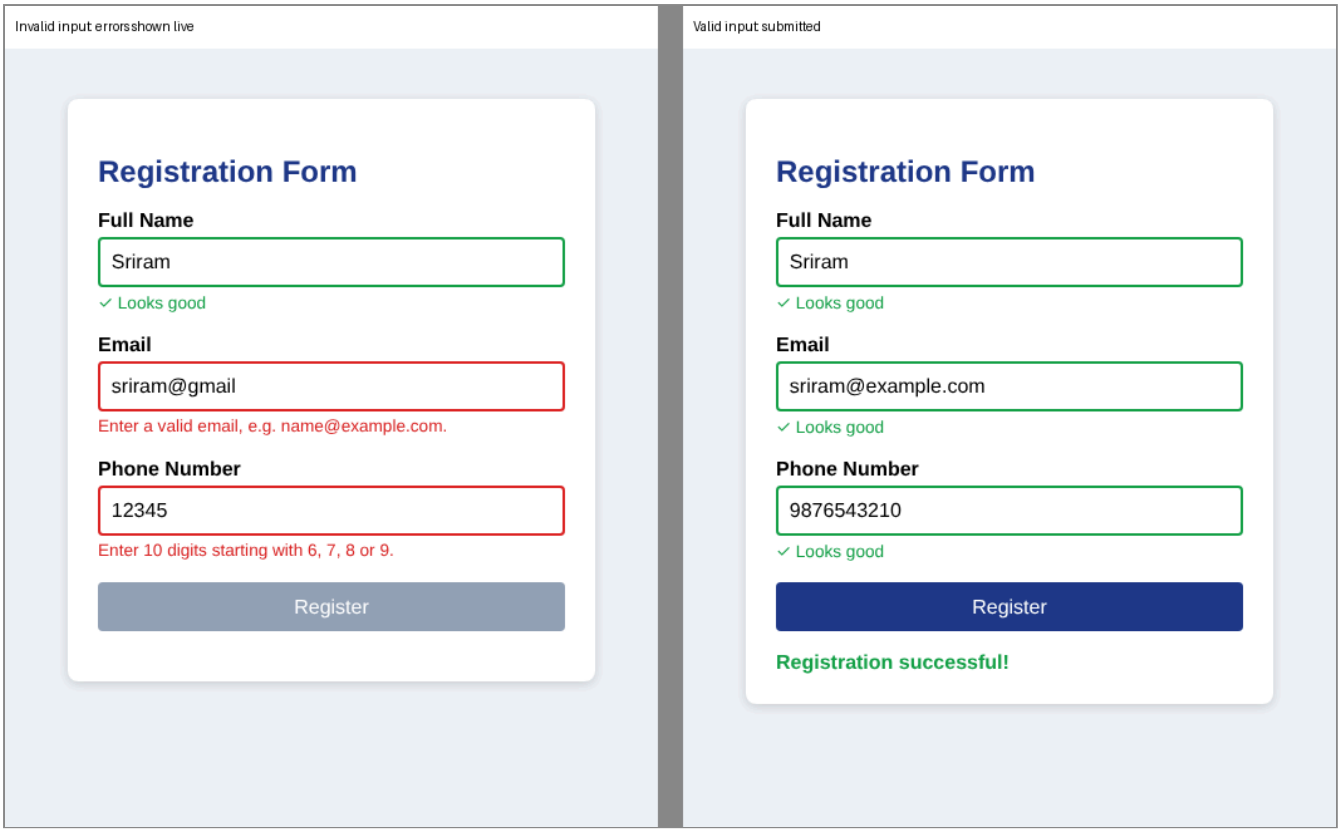
  function updateButton() {
    btn.disabled = !Object.keys(fields).every(k => fields[k].re.test(fields[k].el.value.trim()));
  }

  Object.keys(fields).forEach(key => {
    const f = fields[key];
    f.el.addEventListener('input', () => {
      if (key === 'phone') f.el.value = f.el.value.replace(/\D/g, '');  // digits only
      validate(key, false);
      updateButton();
    });
    f.el.addEventListener('blur', () => validate(key, true));
  });

  document.getElementById('regForm').addEventListener('submit', e => {
    e.preventDefault();
    const allOk = Object.keys(fields).map(k => validate(k, true)).every(Boolean);
    result.textContent = allOk ? 'Registration successful!' : '';
  });
</script>
</body>
</html>
```

Sample input and output

Name	Email (input)	Phone (input)	Output
Sriram	sriram@gmail	12345	Email: *Enter a valid email, e.g. name@example.com.* Phone: *Enter 10 digits starting with 6, 7, 8 or 9.* Register button stays disabled
Sriram	sriram@example.com	98765abcde	Letters are removed as typed, so the field shows 98765 and the phone error appears
Sriram	sriram@example.com	9876543210	All fields show a green tick and **Registration successful!**



Output screenshot: Experiment 2 showing live error messages (left) and successful submission (right)

(e) Evaluation and enhancements

- **Evaluation:** invalid email and phone values are rejected immediately, errors appear as the user types, the submit button is enabled only when all fields are valid, and non-digits are stripped from the phone field.
- **Enhancements:** repeat all checks on the server (client-side checks can be bypassed), sanitize and escape input to prevent XSS/SQL injection, add CAPTCHA or rate limiting, verify email by OTP or confirmation link, use `aria-live` and `aria-describedby` for accessibility, support international phone formats, and add a password-strength meter.

Experiment 3: CSS Layout Debugging

(a) CSS properties responsible for overlapping

- **`position:absolute`** removes elements from normal flow, so they ignore each other and stack on top of one another.
- **Fixed `width` in px** (600px content, 250px sidebar) cannot shrink when the window narrows.
- **Overlapping offsets**: sidebar is 250px wide but the content starts at `left:200px`.
- **Fixed `top` values** (footer at `top:400px`) do not move when content grows, so text overflows into the footer.
- **Content-box sizing**: padding and border are added on top of the declared width, making elements larger than expected.

(b) Role of the CSS box model

Layer (inside to outside)	Role
Content	The text or image area, sized by `width/height`
Padding	Space between content and border; inside the element's background
Border	Line around the padding; adds to the element's total size
Margin	Transparent space outside the border separating neighbouring elements

Total width = content + padding-left/right + border-left/right + margin-left/right. With `box-sizing:border-box`, the declared width already includes padding and border, which makes sizing predictable.

(c) Positioning techniques used for debugging

Technique	Use in debugging
`relative`	Small offset from normal position; also the anchor for absolute children
`absolute`	Only for overlays/badges inside a `relative` parent, never for the page skeleton
Flexbox	One-dimensional rows or columns (menus, toolbars)
Grid	Two-dimensional page layout (header, sidebar, content, footer), used in the fix

Buggy code (exp3_buggy.html)

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Experiment 3 - BUGGY layout (overlaps on resize)</title>
<style>
  body { margin: 0; font-family: Arial, sans-serif; }
  .header { position: absolute; top: 0; left: 0; width: 100%; height: 70px; background: #1e3a8a; color:
#fff; padding: 20px; }
  .sidebar { position: absolute; top: 70px; left: 0; width: 250px; height: 300px; background: #93c5fd;
padding: 10px; }
  .content { position: absolute; top: 70px; left: 200px; width: 600px; background: #fde68a; padding: 20px;
border: 5px solid #92400e; margin: 30px; }
  .footer { position: absolute; top: 400px; left: 0; width: 100%; height: 60px; background: #111827;
color: #fff; padding: 20px; }
</style>
</head>
<body>
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="content">Main content. This text is long enough to wrap onto several lines, and when the
window becomes narrow this block collides with the sidebar and the footer because of fixed widths and
absolute positions.</div>
  <div class="footer">Footer</div>
</body>
</html>
```

(d) Corrected code (exp3_fixed.html)

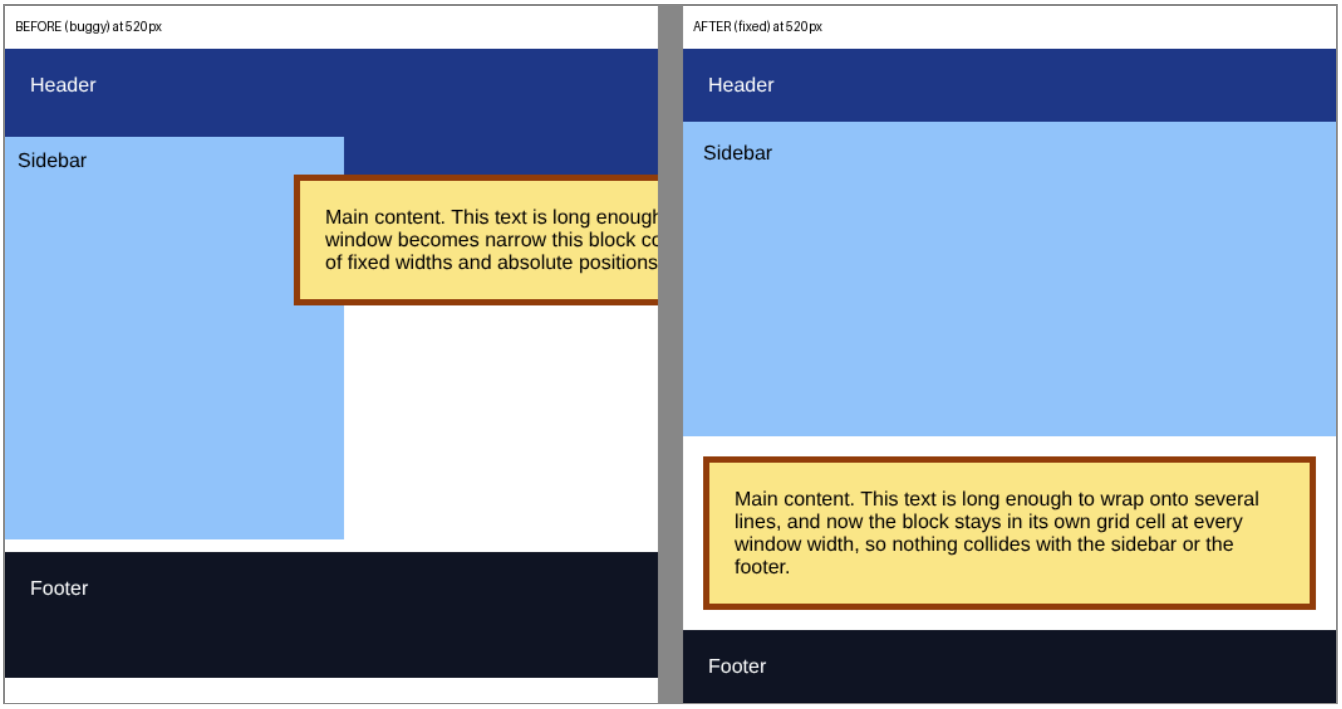
The absolute positions are replaced with a Grid layout using named areas, `box-sizing:border-box`, margins in place of offsets, and a media query that stacks the areas on narrow screens.

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Experiment 3 - FIXED layout</title>
<style>
  *, *::before, *::after { box-sizing: border-box; }   /* width includes padding + border */
  body {
    margin: 0;
    font-family: Arial, sans-serif;
    min-height: 100vh;
    display: grid;                                     /* page skeleton */
    grid-template-rows: auto 1fr auto;
    grid-template-columns: 250px 1fr;
    grid-template-areas:
      "header  header"
      "sidebar content"
      "footer  footer";
  }
  .header  { grid-area: header;  background: #1e3a8a; color: #fff; padding: 1.25rem; }
  .sidebar { grid-area: sidebar; background: #93c5fd; padding: 1rem; }
  .content {
    grid-area: content;
    background: #fde68a;
    padding: 1.25rem;
    border: 5px solid #92400e;
    margin: 1rem;                /* spacing handled by margin, not offsets */
    min-width: 0;                /* lets the grid item shrink */
    overflow-wrap: anywhere;     /* long words never overflow */
  }
  .footer  { grid-area: footer;  background: #111827; color: #fff; padding: 1.25rem; }

  /* Stack everything on small screens */
  @media (max-width: 700px) {
    body {
      grid-template-columns: 1fr;
      grid-template-areas: "header" "sidebar" "content" "footer";
    }
  }
</style>
</head>
<body>
  <div class="header">Header</div>
  <div class="sidebar">Sidebar</div>
  <div class="content">Main content. This text is long enough to wrap onto several lines, and now the
block stays in its own grid cell at every window width, so nothing collides with the sidebar or the
footer.</div>
  <div class="footer">Footer</div>
</body>
</html>
```

Sample input and output

Sample input (window width)	Buggy layout output	Fixed layout output
900px	Content box starts at 200px, overlapping the 250px sidebar	Sidebar and content sit side by side, no overlap
520px	Content overlaps the sidebar and header and runs off screen	Areas stack in one column, no overlap



Output screenshot: Experiment 3 before (left) and after (right) the fix at 520px

(e) Evaluation and best practices

- **Evaluation:** the fixed layout keeps header, sidebar, content and footer in separate grid cells, so nothing overlaps at any window width. Below 700px the areas stack into one column. The before/after screenshot at 520px shows the difference.
- **Best practices:** use `box-sizing:border-box` globally; prefer Flexbox/Grid over absolute positioning; use relative units (%, rem, fr, vw) and `max-width`; use `min-width:0` and `overflow-wrap` for long content; adopt a mobile-first approach with media queries; keep CSS variables for spacing; test with browser DevTools at several widths; and use a CSS reset.